

Yes. For this practical, the **easiest setup is Jenkins on your Windows local machine**, with a simple Flask/FastAPI project in GitHub.

Practical: Setting Up CI/CD with Jenkins

Aim

To set up Jenkins for a Flask/FastAPI application and create a CI/CD pipeline that automatically:

Developer → GitHub → Jenkins → Build → Test → Deploy

Part A — Install Jenkins

1. Install Java

Jenkins requires Java.

You should have **JDK 17 or later** installed.

Open **Command Prompt** and run:

```
java -version
```

You should see something similar to:

```
java version "17..."
```

If Java is working, continue.

2. Install Git

Check Git:

```
git --version
```

If you get something like:

```
git version 2.x.x
```

Git is installed correctly.

3. Install Jenkins

Download Jenkins from the official Jenkins website:

[Jenkins official website](https://jenkins.io/)

For Windows, download the **Windows installer (.msi)**.

During installation:

1. Run the Jenkins installer.
2. Keep the default installation location.
3. Select **LocalSystem** if asked about the service account.
4. Select the Java installation.
5. Keep the default Jenkins port, usually:

8080

6. Complete the installation.
-

4. Open Jenkins

Open your browser and go to:

```
http://localhost:8080
```

You should see:

Unlock Jenkins

Jenkins will ask for an initial administrator password.

The password is normally stored here:

C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword

Open that file using Notepad and copy the password.

Paste it into Jenkins.

5. Install Jenkins plugins

Select:

Install suggested plugins

Wait until the installation finishes.

Then create your Jenkins administrator account.

You should finally reach the Jenkins dashboard.

Part B — Create a Simple Flask Application

Create a folder:

flask-jenkins-demo

Open it in VS Code.

Create:

```
flask-jenkins-demo/  
├── app.py  
├── requirements.txt  
└── test_app.py
```

app.py

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route("/")  
def home():  
    return "Hello from Flask CI/CD!"
```

```
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
Flask  
pytest
```

test_app.py

```
from app import app  
  
def test_home():  
    client = app.test_client()  
    response = client.get("/")  
  
    assert response.status_code == 200  
    assert response.data == b"Hello from Flask CI/CD!"
```

Part C — Test the Application Manually

Open the VS Code terminal.

Create a virtual environment:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Run the test:

```
pytest
```

You should get:

```
1 passed
```

Run Flask:

```
python app.py
```

Open:

`http://localhost:5000`

You should see:

Hello from Flask CI/CD!

Part D — Upload Project to GitHub

Create a new repository on GitHub, for example:

`flask-jenkins-demo`

Then in your VS Code terminal:

```
git init
git add .
git commit -m "Initial Flask application"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

After pushing, your GitHub repository should contain:

```
app.py
requirements.txt
test_app.py
```

Part E — Create Jenkins Pipeline

Go to:

```
Jenkins Dashboard
    ↓
New Item
```

Enter:

Flask-CI-CD

Select:

Pipeline

Click:

OK

Configure Pipeline

Scroll down to:

Pipeline

Select:

Pipeline script

Enter this Jenkinsfile:

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
            url: 'YOUR_GITHUB_REPOSITORY_URL'
      }
    }

    stage('Install Dependencies') {
      steps {
        bat 'python -m pip install -r requirements.txt'
      }
    }

    stage('Test') {
      steps {
        bat 'pytest'
      }
    }

    stage('Deploy') {
      steps {
        bat 'echo Application deployed successfully'
      }
    }
  }
}
```

Replace:

YOUR_GITHUB_REPOSITORY_URL

with your actual GitHub repository URL.

For example:

```
git branch: 'main',  
  url: 'https://github.com/username/flask-jenkins-demo.git'
```

Click:

Save

Part F — Run the Jenkins Pipeline

Open your Jenkins job:

Flask-CI-CD

Click:

Build Now

You will see:

Build #1

Click the build number and select:

Console Output

You should see stages such as:

```
[Pipeline] stage  
[Pipeline] { (Checkout)  
  
[Pipeline] stage  
[Pipeline] { (Install Dependencies)  
  
[Pipeline] stage  
[Pipeline] { (Test)  
  
1 passed  
  
[Pipeline] stage  
[Pipeline] { (Deploy)  
  
Application deployed successfully
```

If everything succeeds, Jenkins will show:

Finished: SUCCESS

Part G — Automatic Build When Code Is Pushed

This is the important part of **CI/CD integration with Git**.

Instead of manually clicking **Build Now**, Jenkins can automatically start whenever you push code to GitHub.

There are two common methods:

Method 1 — GitHub Webhook

The workflow becomes:

```
Developer
  ↓
Changes Code
  ↓
git push
  ↓
GitHub
  ↓
Webhook
  ↓
Jenkins
  ↓
Checkout
  ↓
Install
  ↓
Test
  ↓
Deploy
```

Configure Jenkins

Open:

```
Jenkins
→ Flask-CI-CD
→ Configure
```

Find:

Build Triggers

Select:

GitHub hook trigger for GITScm polling

Save.

Part H — Configure GitHub Webhook

Go to your GitHub repository.

Select:

Settings
→ Webhooks
→ Add webhook

For a Jenkins server that GitHub can reach, enter your Jenkins webhook endpoint:

`http://YOUR_JENKINS_SERVER/github-webhook/`

Select:

Content type:
`application/json`

Choose:

Just the push event

Click:

Add webhook

Important

If Jenkins is running only on your computer at:

`localhost:8080`

GitHub cannot normally send a webhook directly to `localhost`.

For a classroom practical, you can demonstrate the pipeline with **Build Now**. For a real automatic webhook, Jenkins needs to be reachable from GitHub, typically through a properly configured public server/tunnel or hosted Jenkins instance.

Part I — Test Automatic CI/CD

Make a change in `app.py`.

For example:

```
@app.route("/hello")
def hello():
    return "Hello from Jenkins!"
```

Then:

```
git add .
git commit -m "Added hello endpoint"
git push
```

GitHub receives the change.

The webhook triggers Jenkins.

Jenkins automatically:

1. Downloads latest code
2. Installs dependencies
3. Runs tests
4. Deploys application

You can see the new build under:

Jenkins → Flask-CI-CD → Build History

What You Need Installed

For this practical, install/check these:

Software	Purpose
JDK 17+	Required by Jenkins
Jenkins	CI/CD automation
Git	Version control

Software	Purpose
Python	Flask application
VS Code	Development
GitHub account	Remote repository
Flask	Backend application
Pytest	Automated testing

Docker is optional for this basic practical. You don't need Docker just to demonstrate build → test → deploy.

What to Show Your Teacher

For the practical demonstration, show these in order:

1. **Java installed**

```
java -version
```

2. **Git installed**

```
git --version
```

3. **Jenkins dashboard**

```
http://localhost:8080
```

4. **Flask project in VS Code**

5. **GitHub repository**

6. **Jenkins Pipeline configuration**

7. Click **Build Now**

8. Show **Console Output**

9. Show:

```
1 passed
```

10. Show:

```
Finished: SUCCESS
```

11. Make a Git change and `git push`.

12. Demonstrate the new Jenkins build.

Final result

Your practical demonstrates all three requirements:

- a.** Jenkins installed locally 
- b.** Pipeline performs build/install → test → deployment 
- c.** Git/GitHub integrated for automated triggering 